

Car Lease Review & Negotiation

AI Project — Complete Code Explanation

Every file, every line explained in simple words

Project Folder Structure

Your project has 2 main folders that must both run at the same time:

Folder	Technology	Purpose	Start Command
Car-lease-app	Python / FastAPI	Backend brain – reads PDFs, calls AI, does python lookup	python -m uvicorn main:app --r
car-lease-frontend	React (JavaScript)	Frontend face – the website you see in browser	npm start

How Everything Works Together

1. You open localhost:3000 → React app loads in your browser
2. You upload a PDF contract → React sends it to Python API
3. Python reads the PDF → Extracts all text from every page
4. Python sends text to Gemini AI → AI finds APR, payments, mileage etc.
5. Gemini returns JSON data → Python sends results to React
6. React shows the results → You see all contract terms on screen!

Folder 1 — Car-lease-app (Backend / Brain)

This is the Python backend. It runs on port 8000. It does the heavy work — reading PDFs, calling Google Gemini AI, and looking up car details from the government database.

File: vin.py — Car History Lookup

This file's only job is to take a VIN number and return the car's make, model, and year by calling the free US government vehicle database (NHTSA).

```
import requests

# 'requests' = a tool that lets Python call websites/APIs
# Think of it like a browser — it visits a URL and gets the data back

def lookup_vin(vin: str):
    # This is a FUNCTION — like a machine
```

You put a VIN number IN → it gives car details BACK

```
url = f'https://vpic.nhtsa.dot.gov/api/vehicles/DecodeVin/{vin}?format=json'
```

This is the US government website address we call

{vin} = the actual VIN number gets inserted here automatically

```
response = requests.get(url)
```

Go to that website and get the data — like opening a webpage

```
data = response.json()
```

Convert the website response into Python data we can read

```
vehicle_info = {'make': None, 'model': None, 'year': None}
```

Create an empty box with 3 slots: brand name, model name, year

```
for item in data['Results']:
```

Loop through every piece of data the website returned

```
    if item['Variable'] == 'Make':  
        vehicle_info['make'] = item['Value']
```

If we find the brand name (e.g. HONDA) → save it

```
    elif item['Variable'] == 'Model':  
        vehicle_info['model'] = item['Value']
```

If we find the model name (e.g. Accord) → save it

```
    elif item['Variable'] == 'Model Year':  
        vehicle_info['year'] = item['Value']
```

If we find the year (e.g. 2003) → save it

```
    return vehicle_info
```

Send back the completed car info — e.g. {make: HONDA, model: Accord, year: 2003}

File: extractor.py — Contract Text Extraction

This file uses pattern matching (called regex) to search the contract text for specific words like 'APR', 'Monthly Payment', 'Mileage' and grabs the numbers next to them. Note: This is the basic version. The AI version (Gemini) is smarter and handles any format.

```
import re
```

're' = Regular Expressions — a tool to search text using patterns

Like a super-powered CTRL+F that can find patterns, not just exact words

```
def extract_sla_fields(text):
```

This function receives the FULL contract text as one big string

and searches it for important numbers and terms

```
    sla_fields = {  
        'apr_percent': None,
```

```

        'term_months': None,
        'monthly_payment': None,
        'down_payment': None,
        'residual_value': None,
        'mileage_allowance': None,
        ...
    }

# Create empty boxes for each field — all start as None (empty)
# We will fill these boxes as we find each value in the contract

    apr = re.search(r'(APR|Annual Percentage Rate|Interest Rate)[^\d]*([\d\.]+)', text)
# Search the text for words like 'APR' or 'Interest Rate'
# [^\d]* = skip any non-number characters after those words
# ([\d\.]+) = grab the number that follows (e.g. 5.99)

    if apr:
        sla_fields['apr_percent'] = apr.group(2)
# If we found a match → save the number into our apr_percent box
# .group(2) means 'give me the second captured piece' = the number

    return sla_fields

# Send back all the boxes — some filled, some still empty (None)

```

File: main.py — The Main API Server

This is the most important backend file. It creates a web server with 3 URLs (called endpoints). React calls these URLs to upload contracts, look up VINs, and check the server is running.

```

from fastapi import FastAPI, UploadFile, File

# FastAPI = a tool that creates web endpoints (URLs) very easily
# UploadFile = handles PDF files that users send from the browser

from fastapi.middleware.cors import CORSMiddleware
# CORS = Cross-Origin Resource Sharing
# Without this, React (port 3000) cannot talk to Python (port 8000)
# It is a security rule — we turn it off so our own app can work

import PyPDF2
# Tool that reads PDF files and extracts the text from each page

import google.generativeai as genai
# This is Google's Gemini AI library — lets us send text to Gemini

import json
# Tool to convert between Python objects and JSON text format

from vin import lookup_vin
# Import our lookup_vin function from vin.py file

```

[illegible]

[illegible]

Folder 2 — car-lease-frontend (Frontend / Face)

This is the React frontend. It runs on port 3000. It is the website you see in the browser. It has 3 tabs: Contract Upload, VIN Lookup, and Negotiation Chat.

File: src/index.js — App Starter (Ignition Key)

```
import React from 'react';  
  
# Import the React library — needed for all React apps  
  
import ReactDOM from 'react-dom/client';  
  
# ReactDOM = tool that connects React to the actual HTML page  
  
import App from './App';  
  
# Import our main App.js file — this is where all our code lives  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
  
# Find the element in public/index.html  
  
# This is an empty div that we will fill with our React app  
  
root.render();  
  
# Put our entire App component inside that div  
  
# This STARTS everything — the whole app appears on screen!
```

File: src/App.js — The Entire User Interface

Section 1: Imports and Setup

```
import { useState } from 'react';  
  
# useState = a React tool that lets us REMEMBER things
```

Example: remember which tab is currently open
When state changes → screen automatically updates!

```
import axios from 'axios';  
  
# axios = a tool to call our Python API from JavaScript  
# Like fetch() but easier to use
```

```
const API = 'http://127.0.0.1:8000';  
  
# The address of our Python backend server  
# All API calls will go to this address
```

Section 2: State Variables (Memory)

State variables are like the app's memory. When they change, the screen updates automatically.

```
const [tab, setTab] = useState('upload');  
  
# Remember which tab is active: 'upload', 'vin', or 'chat'  
# useState('upload') = start on the upload tab by default  
# setTab(...) = call this to switch tabs  
  
const [file, setFile] = useState(null);  
  
# Remember which PDF file the user selected  
# null = no file selected yet  
  
const [loading, setLoading] = useState(false);  
  
# Remember if we are currently waiting for API response  
# true = show 'Analyzing...' button false = show 'Analyze' button  
  
const [slaResult, setSlaResult] = useState(null);  
  
# Remember the extracted contract terms from the API  
# When this gets set → the results grid appears on screen  
  
const [vin, setVin] = useState('');  
  
# Remember what VIN number the user is typing  
  
const [vinResult, setVinResult] = useState(null);  
  
# Remember the car details returned from VIN lookup  
  
const [messages, setMessages] = useState([]);  
  
# Remember the full chat history (array of messages)  
# Each message has: role ('user' or 'bot') and text
```

Section 3: handleUpload — PDF Upload Function

```
const handleUpload = async () => {  
  
# This function runs when user clicks 'Analyze Contract' button  
# 'async' means it can WAIT for the API to respond  
  
    if (!file) return;  
  
# If no file selected → do nothing, stop here  
  
    setLoading(true);
```

Show the 'Analyzing...' spinner on the button

```
const form = new FormData();  
form.append('file', file);
```

Pack the PDF file into a FormData object

This is the standard way to send files over HTTP

```
const res = await axios.post(`${API}/upload-contract`, form);
```

Send the PDF to our Python API at /upload-contract

'await' = WAIT here until Python responds (don't continue yet)

```
setSlaResult(res.data.sla_fields);
```

Save the extracted fields from Python's response

React automatically re-renders → results grid appears!

```
setLoading(false);
```

Hide the spinner — analysis is complete

```
};
```

Section 4: handleVin — VIN Lookup Function

```
const handleVin = async () => {
```

Runs when user clicks 'Look Up' button on VIN tab

```
const res = await axios.get(`${API}/vin/${vin}`);
```

Call Python API with the VIN number the user typed

Example: GET http://127.0.0.1:8000/vin/1HGCM82633A004352

```
setVinResult(res.data.vehicle_info);
```

Save the car details → vehicle card appears on screen

Example: { make: 'HONDA', model: 'Accord', year: '2003' }

```
};
```

Section 5: sendMessage — Negotiation Chatbot

```
const sendMessage = async (text) => {
```

Runs when user sends a chat message

```
const msg = text || chatInput.trim();
```

Use the quick-button text OR what the user typed

```
setMessages(m => [...m, { role: 'user', text: msg }]);
```

Add user's message to chat history

'...' = spread operator — keep all old messages, add new one

```
const lower = msg.toLowerCase();
```

Convert to lowercase for easier matching

```
if (lower.includes('apr') || lower.includes('interest'))
```

```
  reply = 'APR Tips: A good APR is typically 3-7%...'
```

Rule-based responses — if user asks about APR → give APR advice

Different keywords trigger different helpful responses

```
    setTimeout(() => {
      setMessages(m => [...m, { role: 'bot', text: reply }]);
    }, 800);
    # After 0.8 second delay → add bot reply to chat
    # The delay makes it feel more natural, like AI is thinking
  };
}
```

Section 6: return() — What You See on Screen

The return() section contains JSX — which looks like HTML but is actually JavaScript. Everything inside here is what gets displayed in the browser.

```
return (
  /* The outer wrapper div – full dark background */

  Car Lease Review & Negotiation
  /* The big orange title at the top of the page */

  3 tabs
  setTab('upload')>Contract
  setTab('vin')>VIN Lookup
  setTab('chat')>Negotiate

  /* 3 tab buttons – clicking each one calls setTab() */
  /* setTab() changes the 'tab' state → screen updates */

  {tab === 'upload' && }
  /* IF tab equals 'upload' → show the upload area */
  /* Otherwise → show nothing (hide it) */

  {tab === 'vin' && }
  /* IF tab equals 'vin' → show the VIN input form */

  {tab === 'chat' && }
  /* IF tab equals 'chat' → show the chatbot */

);
```

Daily Commands — How to Start Your Project

Every time you want to work on your project, open 2 CMD windows and run these commands:

CMD Window	Command	What it does
------------	---------	--------------

Window 1 (Backend)	cd C:\Users\pramo\OneDrive\Desktop\Car-lease-app python -m uvicorn main:app --reload	Starts Python API on port 8000
Window 2 (Frontend)	cd C:\Users\pramo\OneDrive\Desktop\car-lease-frontend npm start	Starts React app on port 3000

Then open browser at: <http://localhost:3000>

Quick Troubleshooting Guide

Problem	Fix
Upload failed. Make sure API is running!	Start backend: <code>python -m uvicorn main:app --reload</code>
localhost:3000 — This site can't be reached	Start frontend: <code>npm start</code> (in car-lease-frontend folder)
Found 0 fields from contract	Gemini AI key missing or wrong in main.py — check <code>GEMINI_API_KEY</code>
uvicorn not recognized	Use: <code>python -m uvicorn main:app --reload</code> (add 'python -m' prefix)
npm not recognized	Install Node.js from nodejs.org then restart CMD
ModuleNotFoundError	Run: <code>pip install [module-name]</code> e.g. <code>pip install google-generativeai</code>